

增強課程 6：實作 BAdI (Enhancement Implementation)

Lecture

en05 建好了 Enhancement Spot / BAdI Definition，這題要做的是「實作」——建立真正的 Implementation，讓客製化邏輯真的跑起來。這題準備了兩個案例：自己建的 **ZES_EN05_GREETING** (安全、可控，示範 Multi Use 多實作機制) 與真實標準 BAdI **WORKORDER_UPDATE** (有風險、需要安全閘設計，示範真實企業客製化的完整流程)。

Implementation 建立也是 **GUI-only (SE19)**，跟 en04 的 **Source Code Plugin**、en05 的 **Enhancement Spot** 一樣：直接對 `/sap/bc/adt/enhancements/enhoxh` POST 一路依錯誤訊息補齊 `enho:contentCommon`、`enho:runtimeBehaviorShorttext` 等必要元素後，最後回 `Resource controller does not support method POST`——這個端點本身就不支援 POST，不是資料格式問題，證實建立空殼一定要走 SE19；空殼建好之後的內容讀寫 (含直接指定 Interface 方法邏輯) 可以正常走 ADT。

案例一：**ZES_EN05_GREETING** 加第二個 **Implementation**，驗證 **Multi Use** 執行順序

Multi Use BAdI 沒有 Filter 時，**CALL BADI** 不是「只挑一個 Implementation 執行」，而是依序呼叫所有 **Active** 的 **Implementation**，每一個都拿到前一個處理過的 **CHANGING** 參數繼續往下改——這正是 en05 學到的「Multi Use 不能用 **RETURNING/EXPORTING**」規則存在的原因。本題新增第二個 Implementation **ZIM_EN06_GREETING2** (Class **ZCL_EN06_GREETING2**)，邏輯是把文字追加在 `cv_text` 後面而不是覆蓋。實測結果：

```
Bon voyage on LH! (real implementation ZCL_EN05_GREETING is active) + safe travels,
LH! (2nd implementation ZCL_EN06_GREETING2 also fired)
```

證實兩個 Implementation 依序都執行了 (**ZCL_EN05_GREETING** 先、**ZCL_EN06_GREETING2** 後)，而不是只有一個生效——這是 Multi Use 跟 Single Use 最直觀的行為差異：Single Use 保證「最多一個」，Multi Use 允許「多個同時貢獻結果」。

案例二：真實標準 BAdI **WORKORDER_UPDATE / AT_SAVE**

WORKORDER_UPDATE (套件 **COBADI**，真正的 **ENHS/XS**，Multi Use，Interface **IF_EX_WORKORDER_UPDATE**) 是 PM/PP/PS/PI 訂單存檔時的真實掛勾點，方法 **AT_SAVE** 在存檔時被呼叫：

```
methods AT_SAVE
importing
    !IS_HEADER_DIALOG type COBAI_S_HEADER_DIALOG
exceptions
    ERROR_WITH_MESSAGE .
```

查證 **COBAI_S_HEADER_DIALOG** (**COBAI** Type Group 裡的型別) 發現它 **LIKE CAUFVD**——跟 en04 用過的結構完全一樣，代表可以直接沿用 en04 已驗證的安全閘設計：只有 **IS_HEADER_DIALOG-WERKS = '1011'** 且

IS_HEADER_DIALOG-AUART = 'PP71' (教學專用組合) 才動作，其餘一律不做任何事，不影響任何真實 PM/PP/PS/PI 工單存檔。

⚠ 這題端對端驗證過程中踩到一個嚴重到會讓系統當機的真實錯誤——絕對不能在 BAdI Implementation 裡下 COMMIT WORK：第一版程式碼在寫完稽核記錄後加了一行 COMMIT WORK。(想著「保險起見存進去」)，結果使用者一存檔就整個 Dump：

```
Category          ABAP programming error
Runtime Errors     MESSAGE_TYPE_X
ABAP Program       SAPLCOZV
* Unexpected COMMIT WORK!!!
* there should be no COMMIT WORK in order processing before
* fm CO_ZV_ORDER_POST was executed, since this might lead to
* inconsistencies!!!
```

原因：AT_SAVE 是在訂單存檔框架 (SAPLCOZV) 還沒處理完的交易 (LUW) 中間被呼叫的，這個 LUW 的擁有者是最上層的存檔框架，不是我們的 Implementation。COMMIT WORK 只能由「擁有 LUW 的最上層呼叫者」下達；被呼叫的子程式 / Implementation / Function Module 如果自己下 COMMIT WORK，等於在別人還沒做完事的時候硬生生把交易切斷，可能讓資料庫留在不一致的中間狀態——SAP 標準框架非常清楚這個風險，所以在存檔流程裡主動放了偵測機制，一旦發現不該出現的 COMMIT WORK 就直接讓程式當機，寧可讓開發者馬上發現問題，也不讓不一致的資料悄悄進資料庫。拿掉 COMMIT WORK 之後，稽核記錄的 INSERT 會跟著訂單存檔本身的交易一起被最上層框架 COMMIT，不需要 (也不能) 自己額外處理。

修正、重新啟用、真實 C001 存檔 (Plant 1011 / Order Type PP71) 測試成功，ZEN06_ATSAVE_LOG 正確寫入一筆記錄，AUFNR 欄位顯示 %000000000001——一個附帶的觀察：AT_SAVE 觸發當下，工單號碼似乎還是內部暫時性的編號格式 (% 開頭)，還沒轉成最終的 12 碼格式，這代表在 AT_SAVE 這個時間點，不能假設拿到的工單號碼已經是最終格式，如果客製化邏輯需要用到「正式工單號碼」，可能要考慮換一個更晚的掛勾點 (例如 IN_UPDATE，在 Update Task 階段執行，號碼應該已經確定)。

學習目標

- 能講出 BAdI Implementation 的建立也是 GUI-only (SE19)，跟 Enhancement Spot、Source Code Plugin 同一個模式：空殼建立要 GUI，內容讀寫可以走 ADT
- 能講出 Multi Use BAdI 沒有 Filter 時的執行行為：所有 Active Implementation 依序執行，每個都能在前一個的 CHANGING 結果上繼續處理，不是「只挑一個」
- 能講出「絕對不能在 BAdI Implementation 裡下 COMMIT WORK」的原因：LUW 的擁有權屬於最上層呼叫者，子程式自行 COMMIT 會打斷別人未完成的交易，這是會讓系統直接 Dump 的嚴重錯誤，不是隱性 bug
- 能講出如何查證一個 BAdI 方法的參數型別本質 (如 COBAI_S_HEADER_DIALOG LIKE CAUFVD)，並判斷能不能沿用之前課程已驗證過的安全閘設計
- 知道 BAdI 掛勾點被呼叫的「時間點」會影響能拿到的資料完整度 (例如 AT_SAVE 時工單號碼可能還是暫時格式)，設計客製化邏輯前要先確認掛勾點的執行時機

事前準備 (已於本系統 client 130 實際完成，非假設)

1. 案例一 (承接 en05 物件)：ZIM_EN06_GREETING2 / ZCL_EN06_GREETING2 (\$TMP，使用者於 SE19 建立骨架，Claude 用 ADT 寫入內容)，掛在 en05 的 ZES_EN05_GREETING 底下，邏輯是把文字追加到

cv_text 後面。已用 programrun 無頭執行驗證：ZR_EN05_FLIGHT_GREETING_DEMO (加寬 LINE-SIZE 避免 Classic List 截斷長字串) 顯示兩個 Implementation 依序都執行，ZCL_EN05_GREETING 先、ZCL_EN06_GREETING2 後。

2. 案例二 (真實標準 BAdI)：ZEN06_ATSAVE_LOG (自訂稽核表，\$TMP，WERKS+AUART+AUFNR+LOGDATE+LOGTIME)、ZIM_EN06_WORKORDER_ATSAVE / ZCL_EN06_WORKORDER_ATSAVE (\$TMP，使用者於 SE19 建立骨架，掛在真實標準 Enhancement Spot WORKORDER_UPDATE 底下，Interface IF_EX_WORKORDER_UPDATE)，只在 WERKS='1011' / AUART='PP71' 才寫稽核記錄，其餘方法均為空殼、不做任何事。

3. 兩層驗證：

- 單元測試 (ZR_EN06_ATSAVE_UNIT_TEST，\$TMP)：不透過真實 BAdI 派送，直接 CREATE OBJECT + 呼叫方法，驗證安全閘組合 (1011/PP71) 寫入 1 筆記錄、非安全閘組合 (1011/PP01) 寫入 0 筆記錄，programrun 無頭執行驗證成功。
- 真實存檔測試：使用者用 C001 (Plant 1011 / Order Type PP71) 建立真實工單並存檔，ZEN06_ATSAVE_LOG 正確寫入一筆新記錄 (AUFNR=%00000000001)，證實 Enhancement 對真實訂單存檔確實生效。過程中一度因 Implementation 誤含 COMMIT WORK 導致真實 Dump (MESSAGE_TYPE_X，SAPLCOZV)，已修正並重新驗證成功。

題目需求

1. 解釋 Multi Use 沒有 Filter 時的執行語意：如果有 3 個 Active Implementation 都修改同一個 CHANGING 參數，最終結果會是什麼？這跟「只有一個 Implementation 會生效」的直覺印象有什麼落差？
2. 解釋為什麼 BAdI Implementation 不能下 COMMIT WORK，並說明如果拿掉安全機制 (假設 SAP 沒有 SAPLCOZV 那段偵測邏輯)，偷偷下 COMMIT WORK 會造成什麼樣的資料風險 (提示：想想如果存檔框架後面還有其他表格要更新，你的 COMMIT WORK 提前把交易切斷會發生什麼事)。
3. 解釋為什麼要先做單元測試 (直接呼叫 Class 方法)，再做真實存檔測試：這個順序具體避免了什麼風險？如果跳過單元測試直接用真實訂單測試安全閘邏輯，有沒有可能造成本題實際發生過的那種當機？
4. AUFNR 在 AT_SAVE 時顯示 %00000000001 這個觀察，對設計 BAdI 客製化邏輯有什麼提醒：如果你的需求是「工單存檔後，把正式工單號碼寫進另一張表」，還適合用 AT_SAVE 這個掛勾點嗎？

參考答案

Multi Use 無 Filter 的執行語意：3 個 Active Implementation 會依 SAP 決定的順序 (通常是建立順序，但不保證，且沒有官方文件承諾特定順序) 依序執行，每一個都收到前一個處理完的 CHANGING 參數值繼續處理——最終結果是三個 Implementation 的效果疊加 / 串接，不是「三選一」。這跟直覺常見的「BAdI 只會有一個生效」印象 (多半是從 Single Use 或有 Filter 篩選的情境建立的印象) 不同：只要是 Multi Use 且沒有 Filter 區隔，所有 Active Implementation 都會被呼叫，設計 Implementation 時要考慮到「別人的 Implementation 也可能同時在跑」，不能假設自己是唯一的客製化。

為什麼不能 COMMIT WORK：BAdI Implementation 是被別的程式 (存檔框架) 呼叫的一段邏輯，執行當下是在框架的 LUW 裡面，框架可能後面還有其他表格更新、其他一致性檢查要做。如果 Implementation 自己下 COMMIT WORK，等於把「目前已經做的變更」提前釘死，框架後續如果因為某個原因需要 ROLLBACK (例如後面某個檢查失敗)，已經被提前 COMMIT 的部分沒辦法再撤銷——資料庫會停在「部分完成、部分沒完成」的不一致狀態 (例如工單頭已存但工序資料還沒存，或反過來)。SAP 在 SAPLCOZV 裡主動偵測這個風險並讓程式直接 Dump，是刻意設計成「寧可立刻讓開發者發現、也不讓不一致資料默默進資料庫」。

單元測試先行的理由：直接呼叫 Class 方法測試 (不透過真實 BAdI 派送) 完全不會觸碰到真實訂單存檔的 LUW / 交易框架，即使邏輯有 bug (例如本題原本誤含的 COMMIT WORK)，最多就是單元測試本身的

`programrun` 執行結果不符預期，不會影響任何真實資料或觸發框架層級的當機——因為單元測試呼叫時根本沒有一個「框架的 LUW」存在。本題如果跳過單元測試、直接拿真實訂單測試（其實本題真的是這樣做才踩到 Dump 的，單元測試是*之後*才補上的防護），代價就是使用者的 CO01 交易直接當機，雖然 ABAP Dump 會自動 ROLLBACK 不會留下壞資料，但仍然是一次不必要的、會嚇到使用者的失敗經驗——**先單元測試、確認邏輯正確，再碰真實交易**，是能大幅降低這類風險的順序。

AT_SAVE 時機的提醒：`AUFNR=%000000000001` 這種格式顯示工單號碼在 **AT_SAVE** 當下可能還沒轉成最終格式（%開頭像是內部暫時性編號的記號）。如果需求是「工單存檔後，把正式工單號碼寫進另一張表」，**AT_SAVE** 可能不是最適合的掛勾點——應該改用 **IN_UPDATE**（在 Update Task 階段執行，通常在這個時間點主要的號碼分配都已經確定）或是先驗證 **AT_SAVE** 拿到的號碼在特定情境下是否已經是最終格式（不同的訂單類型/建立情境可能行為不同，不能一概而論）。這是一個很好的提醒：**BAdI 文件通常只講「什麼時候被呼叫」**，不會明講「這個時間點資料完整到什麼程度」，遇到不確定的欄位，實際測試觀察比看文件猜測更可靠。

思考題

1. 本題案例一（`ZES_EN05_GREETING`）跟案例二（`WORKORDER_UPDATE`）都是 Multi Use，但案例一我們刻意設計成「兩個 Implementation 都追加文字，順序清楚可見」，案例二則是「只有一個 Implementation，安全閘決定要不要動作」。如果案例二也想做成「多個 Implementation 依序處理」的設計（例如一個負責記錄稽核、另一個負責額外驗證），要注意什麼？（提示：想想如果兩個 Implementation 都要用同一個安全閘條件、但各自獨立開發維護，會不會有條件重複維護、容易漏改其中一處的風險——這是多 Implementation 情境下常見的維護痛點）
2. **COMMIT WORK** 的教訓具體發生在「訂單存檔框架」這個情境，但這個規則其實適用於**所有**被別人呼叫的程式碼（Function Module、Method、BAdI Implementation.....）。回想 en02 的 Number Range 教訓（`NUMBER_GET_NEXT` 取號非交易性、不受 COMMIT/ROLLBACK 約束）——這兩個「交易正確性」的教訓分別提醒你什麼不同的風險？（提示：Number Range 的教訓是「即使你 ROLLBACK，某些系統動作仍然生效、不會復原」；這題的教訓是「你自己主動 COMMIT，可能讓別人還沒做完的事被迫提前定型」——兩者都是「交易邊界不是你以為的那樣」，但方向相反）
3. 本題的安全閘設計（`WERKS='1011'` 且 `AUART='PP71'`）跟 en04 用的是同一組測試值。如果之後 en07（Explicit Enhancement Point）或期末綜合實作也需要用到「真實但安全」的測試資料，你會建議繼續沿用這組 `1011/PP71`，還是每題各自另外找一組？為什麼？（提示：沿用同一組的好處是「已知安全、不用每次重新確認」；缺點是如果之後不同题目的 Enhancement 同時掛在同一個廠別/製令類型組合上，彼此可能互相干擾、難以個別驗證——需要視題目之間會不會同時生效來決定）